

Software Development Fundamentals (SDF) – I

ODD 2023



File Handling

**Jaypee Institute of Information Technology (JIIT)
, Noida**

Outline

- Lecture 37-38** Introduction to File, Creation of files in C programming language, Modes of File Handling like read, write, update, Different types of files like binary file and text file
- Lecture 39-40** File operations like, opening, closing, reading, writing, end of file, traversing the file, for structured and unstructured data

Lecture 37-38

File Management in C
Opening a File
Modes of File Handling
Closing Files

File

- A named collection of data, stored in secondary storage (typically).
- Typical operations on files:
 - ✓ Open
 - ✓ Read
 - ✓ Write
 - ✓ Close
- How is a file stored?
 - ✓ Stored as sequence of bytes, logically contiguous (may not be physically contiguous on disk).

- The last byte of a file contains the end-of-file character (EOF), with ASCII code 1A (hex).
- While reading a text file, the EOF character can be checked to know the end.
- Two kinds of files:
 - ✓ Text :: contains ASCII codes only
 - ✓ Binary :: can contain non-ASCII characters Image, audio, video, executable, etc.

Opening a File

- In C we use FILE * to represent a pointer to a file.
- fopen is used to open a file. It returns the special value NULL to indicate that it is unable to open the file.

```
FILE *fp; /*variable fp is pointer to type FILE*/
```

```
fp = fopen("filename", "mode"); /*opens file with name filename , assigns identifier to fp */
```

```
if (fp == NULL) {  
    printf ("ERROR IN FILE CREATION");  
}
```

Modes for opening files

The second argument of `fopen` is the *mode in which* we open the file.

There are three modes.

- ✓ "r" opens a file for reading.
- ✓ "w" creates a file for writing, and writes over all previous contents (deletes the file so be careful!).
- ✓ "a" opens a file for appending – writing on the end of the file.

Modes for binary file

We can add a "b" character to indicate that the file is a *binary file*.

- "rb", "wb" or "ab"
- `fptr = fopen ("xyz.jpg", "rb");`

Additional modes

- **r+** open to beginning for both reading/writing
- **w+** same as w except both for reading and writing
- **a+** same as 'a' except both for reading and writing

Closing a file

- File must be closed as soon as all operations on it completed.
- We can close a file simply using `fclose()` and the file pointer.

Syntax: `fclose(file_pointer);`

```
FILE *fp; /*variable fp is pointer to type FILE*/
```

```
fp = fopen("filename", "mode"); /*opens file with name filename , assigns identifier to fp */
```

```
if (fp == NULL) {  
printf ("Error in file opening");  
return -1;  
}
```

```
/* code for file manipulation*/
```

```
fclose (fptr); /*close the file*/
```

Lecture 39-40

Input/Output operations on files
Random Access to Files
Error Handling

C provides several different functions for reading/writing

- ✓ `getc()` – read a character
- ✓ `putc()` – write a character
- ✓ `fprintf()` – write set of data values
- ✓ `fscanf()` – read set of data values
- ✓ `fgets()` – read line from the file

Writing to a file using fprintf()

- fprintf() works just like printf() except that its first argument is a file pointer.

fprintf (fptr, "Hello World!\n"); □ writes **Hello World** in the file

fprintf (fptr, "%d %d", a, b); □ writes the values of variables a and b in the file

Reading Data Using fscanf()

- We can read data from a file using fscanf().

`fscanf(f2, "%d %f", &i, &f);` □ reads values from the file and store it in variables i and f

- fscanf returns EOF when end-of-file reached

Example 1 : fprintf() and fscanf()

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, sum=0;
```

```
    FILE *f2;
```

```
    /* open files in writing mode*/
```

```
    f2 = fopen("data.txt","w");
```

```
    /* write integers to the files*/
```

```
    for(i=10;i<15;i++)
```

```
    {
```

```
        fprintf(f2,"%d\n",i);
```

```
    }
```

```
    fclose(f2);
```

```
    /* open files in reading mode*/
```

```
    f2 = fopen("int_data.txt","r");
```

```
    /* reading integers from the files and adding them*/
```

```
    while(fscanf(f2,"%d",&i)!=EOF)
```

```
    {
```

```
        sum=sum+i;
```

```
        printf("text file: i=%d\n",i);
```

```
    }
```

```
    printf("text sum=%d\n",sum);
```

```
    fclose(f2);
```

```
    return 0;
```

```
}
```

Reading and Writing a character

- Handle one character at a time like `getchar()` and `putchar()`

Writing a character to the file:

- syntax: `putc(c,fp1);`
 - `c` : a character variable
 - `fp1` : pointer to file opened with mode **w**

Reading a character from the file:

- syntax: `c = getc(fp2);`
 - `c` : a character variable
 - `fp2` : pointer to file opened with mode **r**
- file pointer moves by one character position after every `getc()` and `putc()`
- `getc()` returns end-of-file marker EOF when file end reached

Example 2: putc() and getc()

```
#include <stdio.h>

int main()
{
    FILE *fp1;
    char c;

    /* open file for writing */
    f1= fopen("INPUT.txt", "w");

    /*get char from keyboard until CTL-Z*/
    while((c=getchar()) != EOF)
        putc(c,f1);
    fclose(f1);
```

```
/* reopen file in reading mode*/
f1=fopen("INPUT.txt", "r");

/*read character from file INPUT*/
while((c=getc(f1))!=EOF)
/* print the read character to screen */
    printf("%c", c);

    fclose(f1);
return 0;
}
```

Reading lines from a file using fgets()

Syntax:

char *fgets(char *str, int n, FILE *stream)

- str : Pointer to an array of chars where the string read is copied.
- n : Maximum number of characters to be copied into str (including the terminating null-character).
- *stream : Pointer to a FILE object that identifies an input stream.
- returns : the function returns str

Example 3 fgets()

```
#include <stdio.h>
int main()
{
    FILE *fptr;
    char line [1000];
    /* Open file and check it is open */
    while (fgets(line,1000,fptr) != NULL)
    {
        printf ("Read line %s\n",line);
    }
    return 0;
}
```

Writing lines to files using fputs()

- The C library function **int fputs(const char *str, FILE *stream)** writes a string to the specified stream up to but not including the null character.

Declaration

Following is the declaration for fputs() function.

```
int fputs(const char *str, FILE *stream)
```

Parameters

- **str** – This is an array containing the null-terminated sequence of characters to be written.
- **stream** – This is the pointer to a FILE object that identifies the stream where the string is to be written.

Example 4 fputs()

```
#include <stdio.h>
int main () {
FILE *fp;
fp = fopen("file.txt", "w+");
fputs("This is c programming.", fp);
fputs("This is a system programming language.", fp);
fclose(fp);
return(0);
}
```

Random access to files

- How to jump to a given position (byte number) in a file without reading all the previous data?
- ✓ Setting a desired position in the file: **fseek()**
- ✓ Getting the current position in the file: **ftell()**
- ✓ Setting the position at the beginning point: **rewind()**

`fseek (file-pointer, offset, position);`

- position: 0 (beginning), 1 (current), 2 (end)
- offset: number of locations to move from position

Example:

```
fseek(fp,-m, 1); /* move back by m bytes from current position */  
fseek(fp,m,0); /* move to (m+1)th byte in file */
```

Example 5 fseek()

```
#include <stdio.h>

int main ()
{
FILE *fp;
fp = fopen("file.txt","w+");
fputs("This is SDF-1 class", fp);
fseek( fp, 7, SEEK_SET );
fputs(" C Programming Language", fp);
fclose(fp);
return(0);
}
```

Initially program creates the file and writes

This is SDF-1 class but later we had reset the write pointer at 7th position from the beginning and used puts() statement which over-write the file with the following content –

This is C Programming Language

Error handling

- Given file-pointer, we need to check
 - if EOF reached,
 - if error occurred while handling file
 - if problems occur while opening file etc.

Check if EOF reached: feof()

feof() takes file-pointer as input, returns nonzero if all data read and zero otherwise

```
if(feof(fp))  
    printf("End of data\n");
```

Check if any error occur while file handling

ferror() takes file-pointer as input, returns nonzero integer if error detected else returns zero

```
if(ferror(fp) !=0)  
    printf("An error has occurred\n");
```

Error while opening file

- if file cannot be opened then fopen returns a NULL pointer
- Good practice to check if pointer is NULL before proceeding

```
fp = fopen("input.txt", "r");
```

```
if (fp == NULL)  
    printf("File could not be opened \n ");
```

Example 6

```
#include <stdio.h>

int main ()
{ FILE *fp;
  int c;
  fp = fopen("file.txt","r");
  if(fp == NULL)
  {
    printf("Error in opening file");
    return(-1);
  }
```

```
while(1) {
  c = fgetc(fp);
  if( feof(fp) )
    { break ;
  }
  printf("%c", c);
}
fclose(fp);
return(0);
}
```

Example 7

```
#include <stdio.h>

int main ()
{
    FILE *fp;
    char c;
    fp = fopen("file.txt", "w");
    c = fgetc(fp);
    if( ferror(fp) )
    {
        printf("Error in reading from file : file.txt\n");
    }
    clearerr(fp);
    fclose(fp);
    return(0);
}
```